

DQN-ILS: Deep Q-Network para Iterated Local Search no TSP

Leonardo Assumpção, Gabriel Souto

Lucas Barbosa, Matheus Bastos, Rafael Dias

PESC/COPPE/UFRJ – OTIMIA (COS887)

Dezembro/2025

Sumário

1 Introdução

2 Fundamentos

3 Modelagem MDP

4 Arquitetura

5 Metodologia

6 Experimentos

7 Conclusões

Motivação

- O **TSP** (Traveling Salesman Problem) é NP-difícil com vasta aplicação prática.
- **Iterated Local Search (ILS)** combina perturbações e buscas locais.
- Múltiplas estratégias disponíveis: qual usar em cada momento?

Pergunta central: Podemos usar **Deep Reinforcement Learning** para aprender *qual combinação* de estratégias funciona melhor em cada situação?

Evolução: Q-Learning tabular → **Deep Q-Network (DQN)**
Estado contínuo, aproximação por rede neural, experience replay.

Objetivos

- 1 Propor uma abordagem híbrida **DQN-ILS** que integra Deep Q-Network ao framework do Iterated Local Search.
- 2 **Aprender** a partir de episódios de treino qual combinação (perturbação + busca local) tende a melhorar soluções.
- 3 **Avaliar** se a política aprendida generaliza para instâncias não vistas.
- 4 **Comparar** Standard DQN vs Double DQN.

Foco em reduzir o *gap* relativo ao baseline de forma automática.

Sumário

- 1 Introdução
- 2 Fundamentos**
- 3 Modelagem MDP
- 4 Arquitetura
- 5 Metodologia
- 6 Experimentos
- 7 Conclusões

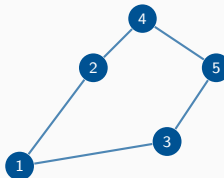
O Problema: TSP

Traveling Salesman Problem:

- Dado um conjunto de n cidades e distâncias d_{ij} , encontrar o **tour de menor custo**.

$$\min_{\pi} \sum_{i=1}^n d_{\pi(i), \pi(i+1)} \quad (\pi(n+1) = \pi(1))$$

- Complexidade: **NP-difícil**
- Soluções exatas: impraticáveis para n grande
- Alternativa: heurísticas e meta-heurísticas



Exemplo: tour com

5 cidades

Iterated Local Search (ILS)

Estrutura básica:

- ① Gerar solução inicial s_0
- ② Aplicar busca local:
 $s \leftarrow \text{BL}(s_0)$
- ③ Repetir até limite de tempo:
 - Perturbar: $s' \leftarrow \text{Pert}(s)$
 - Refinar: $s'' \leftarrow \text{BL}(s')$
 - Aceitar se melhora
- ④ Retornar melhor solução

Componentes intercambiáveis:

Múltiplas perturbações e buscas locais podem ser combinadas.

DQN aprende a escolher dinamicamente!

Deep Q-Network (DQN)

Principais características:

- Aproxima $Q(s, a)$ com rede neural
- **Experience Replay**: armazena transições (s, a, r, s') em buffer
- **Target Network**: rede separada para estabilidade
- Treino off-policy com mini-batches

Update:

$$L = \mathbb{E} [(y - Q_{\theta}(s, a))^2]$$

Standard DQN:

$$y = r + \gamma \max_{a'} Q_{\theta-}(s', a')$$

Double DQN:

$$a^* = \arg \max_{a'} Q_{\theta}(s', a')$$

$$y = r + \gamma Q_{\theta-}(s', a^*)$$

Reduz superestimação de Q-values

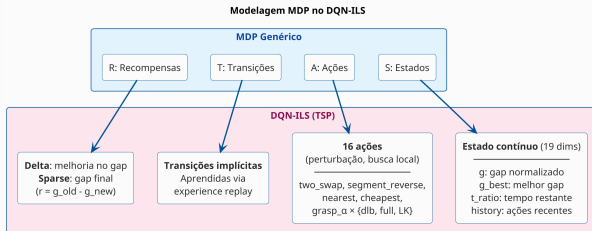
Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP**
- 4 Arquitetura
- 5 Metodologia
- 6 Experimentos
- 7 Conclusões

Modelagem como MDP

MDP: Processo de Decisão de Markov (S, A, T, R)

- **Estado** s : vetor contínuo de 19 dimensões
- **Ação** a : combinação (perturbação, busca local)
- **Transição** T : aprendida implicitamente via experience replay
- **Recompensa** R : baseada em melhoria do gap



Representação de Estado

Estado contínuo (19 dimensões):

Componente	Descrição	Dimensão
g	Gap atual normalizado	1
g_{best}	Melhor gap do episódio	1
t_{ratio}	Tempo restante / T	1
history	Últimas ações (one-hot)	16
Total		19

Normalização do gap: $\text{norm}(g) = \text{sign}(g) \cdot \frac{\log(1+|g|)}{\log(101)}$

Transforma escala logarítmica, simétrico para gaps negativos.

Espaço de Ações

16 ações = combinações de perturbação + busca local:

Ação	Perturbação	Busca Local	Característica
0-2	two_swap	dlb, full, LK	Perturbação leve
3-4	segment_reverse	dlb, full	Perturbação leve
5-7	nearest	dlb, full, LK	Restart com NN
8-9	cheapest	dlb, full	Restart com CI
10-11	grasp_0.03	dlb, full	GRASP quase-guloso
12-13	grasp_0.1	dlb, full	GRASP balanceado
14-15	grasp_0.3	dlb, full	GRASP diversificado

Variantes de 2-opt:

dlb: Don't Look Bits (rápido) | **full:** $O(n^2)$ (melhor) | **LK:** Lin-Kernighan

Função de Recompensa

Delta Reward (padrão):

$$r = g_{\text{best}}^{\text{old}} - g_{\text{best}}^{\text{new}}$$

- Positiva apenas quando melhora o recorde
- Recompensa incremental por step
- Sinal mais frequente durante treino

Sparse Reward:

$$r = \begin{cases} 0 & \text{durante episódio} \\ -\text{norm}(g_{\text{best}}) & \text{no final} \end{cases}$$

- Recompensa apenas no fim
- Menor gap $\rightarrow$ maior reward
- Credit assignment mais difícil

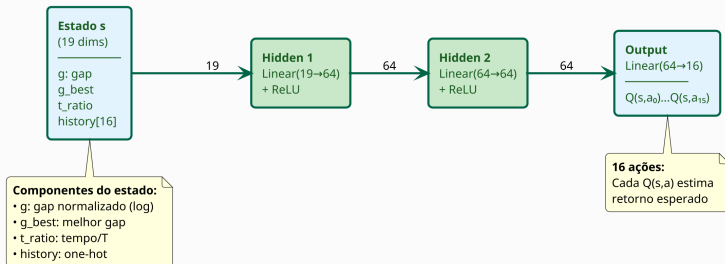
Baseline: Gap calculado relativo a GRASP+2opt (não ao ótimo).
Garante distribuição consistente entre treino e teste.

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Arquitetura**
- 5 Metodologia
- 6 Experimentos
- 7 Conclusões

Arquitetura da Q-Network

Q-Network: Arquitetura MLP (~6.5K parâmetros)



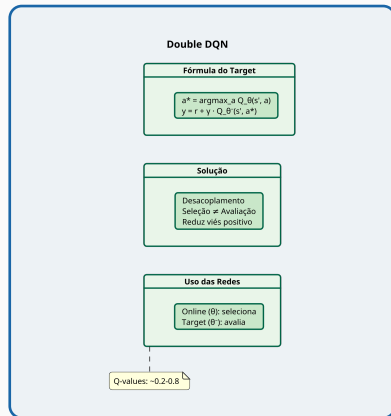
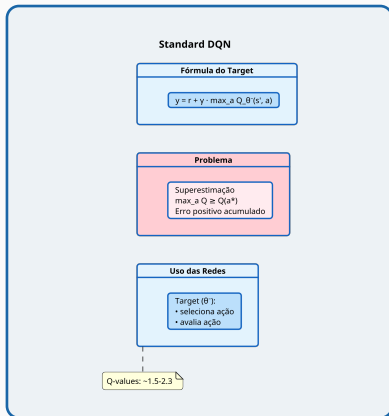
Arquitetura MLP:

Input (19) $\rightarrow$ Linear(64) $\rightarrow$ ReLU
 $\rightarrow$ Linear(64) $\rightarrow$ ReLU $\rightarrow$ Linear(16)

Características:

~6.5K parâmetros total
Leve e eficiente para inferência

Standard DQN vs Double DQN

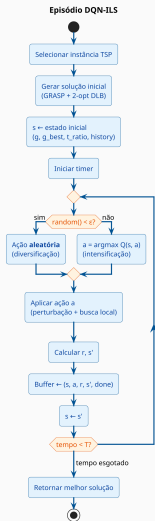


Double DQN desacopla seleção (online) de avaliação (target), reduzindo superestimação.

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Arquitetura
- 5 Metodologia**
- 6 Experimentos
- 7 Conclusões

Fluxo de um Episódio e Espaço de Ações



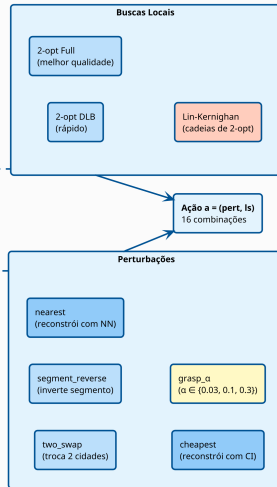
Espaço de Ações: 16 combinações (perturbação × busca local)

Trade-off:

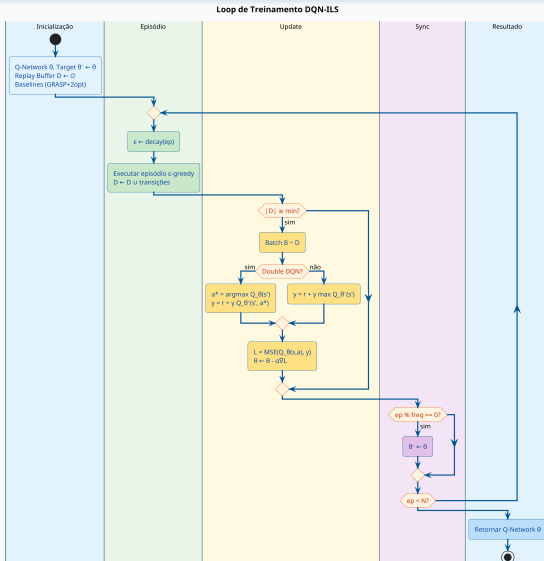
- dlb: $O(n \cdot k)$, mais rápido
- full: $O(n^2)$, melhor solução
- LK: variável, mais potente

Tipos:

- Leves: two_swap, segment_reverse
- Restart: nearest, cheapest
- GRASP: diversificação controlada



Loop de Treinamento



Sincronização da Target Network

Duas redes no DQN:

- **Online** (θ): atualizada a cada batch
- **Target** (θ^-): calcula targets y , atualizada *periodicamente*

Hard update (DQN clássico): $\text{ep}\% \text{freq} = 0$

$$\theta^- \leftarrow \theta \quad (\text{cópia total, freq}=16 \text{ ep})$$

Soft update (Polyak averaging, DDPG):

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^- \quad (\tau \approx 0.005)$$

Atualização gradual a cada passo.

Mais estável, sem tuning de período.

Por que não atualizar sempre?
Se a target mudasse junto com a online, os targets y seriam **alvos móveis**.

⇒ Instabilidade no treinamento

A separação temporal dá à rede online um **alvo fixo**, estabilizando o aprendizado.

Nossa implementação: hard update (padrão DQN).

Hiperparâmetros

Parâmetro	Valor	Descrição
γ	0.99	Discount factor
α (lr)	0.0003	Taxa de aprendizado
hidden_dim	64	Neurônios por camada
batch_size	64	Tamanho do batch
buffer_size	50000	Capacidade do replay buffer
target_update	16	Episódios entre updates
ϵ_{start}	1.0	Diversificação inicial
ϵ_{end}	0.05	Diversificação final

ϵ -decay logarítmico: $\epsilon(t) = \epsilon_{\text{start}} \cdot \left(\frac{\epsilon_{\text{end}}}{\epsilon_{\text{start}}} \right)^{t/T}$

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Arquitetura
- 5 Metodologia
- 6 Experimentos**
- 7 Conclusões

Procedimento Experimental

Instâncias:

- Tipos: EUC_2D, ATT, GEO
- Tamanhos: 10 a 100 vértices
- ~11110 por tipo
- Split: 80% treino, 10% val, 10% teste

Métricas:

- Gap percentual ao ótimo
- Gap percentual ao baseline
- Q-values (monitoramento)
- Distribuição de ações

Ambiente:

- Python + PyTorch
- Treinamento paralelo

Resultado Principal

Conclusão central: O baseline GRASP+2opt é **consistentemente superior** ao DQN-ILS.

Métrica	GRASP+2opt	DQN	Double DQN
Gap médio vs ótimo	0.10%	6.63%	3.99%
% encontrando ótimo	88.7%	6.6%	15.8%
Vitórias head-to-head	92.7%	0.7%	0.8%

Dados: 600 instâncias de teste, EUC_2D, n=20-70, 300 episódios de treino.

Interpretação: O baseline multi-start é extremamente eficiente para TSP de pequeno/médio porte. O DQN-ILS não conseguiu aprender uma política superior.

Resultados por Tamanho de Instância

n	GRASP+2opt		DQN		Double DQN	
	Gap	% ópt	Gap	% ópt	Gap	% ópt
20	0.19%	93%	4.11%	21%	3.52%	33%
30	0.19%	86%	6.51%	6%	6.53%	4%
40	0.04%	97%	3.96%	5%	4.18%	5%
50	0.01%	98%	9.36%	1%	1.19%	35%
70	0.06%	70%	9.20%	0%	4.55%	2%
100	0.30%	19%	8.11%	0%	6.31%	0%

Baseline:

Quase ótimo para $n \leq 50$
(86–98% de taxa de sucesso)

DQN Standard:

Gap cresce muito em $n \geq 50$
(superestimação de Q-values)

Standard DQN vs Double DQN: Q-values

Evolução dos Q-values (n=70):

	Standard	Double
Q_mean início	0.04	0.04
Q_mean final	1.48	0.56
Q_max final	2.31	0.82

Standard DQN: Q-values explodem
(superestimação).

Impacto na qualidade (n=50):

	Standard	Double
Gap médio	9.36%	1.19%
% ótimo	1%	35%

Double DQN > Standard
Melhoria de 8pp no gap (n=50).
Mas perde para baseline.

Por que o DQN-ILS falha?

Hipótese 1: Problema “fácil demais”

- GRASP+2opt multi-start é quase ótimo para $n \leq 100$
- Espaço de busca pequeno o suficiente para heurísticas
- RL tem overhead sem vantagem compensatória

Hipótese 2: Estado pobre

- Estado: $(g, g_{\text{best}}, t, \text{hist})$
- **Não inclui estrutura do tour**
- Rede não sabe *quando* perturbar vs reiniciar

Hipótese 3: Ações mal balanceadas

- 16 combinações com efeitos similares
- Difícil distinguir qual operador é melhor
- Política degenera (ex: 72% mesma ação)

Hipótese 4: Reward inadequado

- Delta reward: só premia ao melhorar recorde
- Maioria das ações $\rightarrow$ reward ≈ 0
- Credit assignment difícil

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Arquitetura
- 5 Metodologia
- 6 Experimentos
- 7 Conclusões**

Discussão: O que Aprendemos

Contribuições técnicas:

- Framework DQN-ILS completo
- Comparação Standard vs Double DQN
- Análise de reward shaping
- Pipeline reprodutível

O que funcionou:

- Double DQN reduz superestimação
- Generalização entre instâncias
- Baseline GRASP+2opt muito forte

O que não funcionou:

- DQN-ILS não supera baseline
- Estado agregado insuficiente
- Overhead de inferência neural
- Escalabilidade limitada

Lição principal:

Para TSP $n \leq 100$, heurísticas multi-start são difíceis de superar com RL de alto nível.

Conclusões

- O **DQN-ILS** não é competitivo com GRASP+2opt para TSP de 20-100 nós.
- **Double DQN** > **Standard DQN**: reduz superestimação, melhora estabilidade.
- O baseline GRASP+2opt é **surpreendentemente forte**: encontra ótimo em 88% dos casos.

Isso não significa que RL é inadequado para TSP:

- A formulação MDP atual (estado agregado, ações de alto nível) não captura informação suficiente
- Para instâncias pequenas, heurísticas simples são difíceis de superar
- Abordagens *end-to-end* (construir tour via RL) podem ser mais promissoras

Direções Futuras

Se persistir com DQN-ILS:

- Estado mais rico: features estruturais do tour (cruzamentos, distribuição de arestas)
- Espaço de ações simplificado: menos combinações, mais distintas
- Curriculum learning: instâncias progressivamente maiores

Alternativas mais promissoras:

Abordagem	Vantagem
PPO, A2C	Rewards esparsos
GNN + RL	Tour como grafo
Attention	Estado da arte

Pointer Networks (Vinyals et al., 2015) e Attention Model (Kool et al., 2019) constroem tours end-to-end.

Referências

- Mnih, V. et al. (2015). *Human-level control through deep reinforcement learning*. Nature. [doi]
- van Hasselt, H., Guez, A., Silver, D. (2016). *Deep Reinforcement Learning with Double Q-learning*. AAAI. arXiv:1509.06461.
- Lillicrap, T. et al. (2016). *Continuous control with deep RL* (DDPG, Polyak avg). ICLR. arXiv:1509.02971.
- Vinyals, O. et al. (2015). *Pointer Networks*. NeurIPS. arXiv:1506.03134.
- Kool, W., van Hoof, H., Welling, M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR. arXiv:1803.08475.
- Bengio, Y., Lodi, A., Prouvost, A. (2020). *Machine Learning for Combinatorial Optimization*. arXiv:1811.06128.
- Sutton, R. S., Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
- OpenAI (2018). *Spinning Up in Deep RL*. spinningup.openai.com.

Agradecimentos



Amor, Carinho e Dedicação

Obrigado!